

- (a) Suppose you are given a triangulation with v vertices, h of which lie on the outer face. Use Euler's formula to derive the number of triangles t and the number of edges e in the triangulation as functions of v and h .

Solution: There's always one more face than there are triangles because of the external face:

$$f = t + 1$$

Euler's formula:

$$v - e + f = 2$$

$$v - e + t + 1 = 2$$

$$v - e + t = 1$$

$$2v - 2e + 2t = 2$$

Total number of edges incident to all triangles:

$$3t = 2e - h$$

$$2e = 3t + h$$

Replace $2e$ with $3t + h$ in the equation $2v - 2e + 2t = 2$:

$$2v - 3t - h + 2t = 2$$

$$2v - t - h = 2$$

$$t = 2v - h - 2$$

Plug t back into $v - e + t = 1$:

$$v - e + 2v - h - 2 = 1$$

$$3v - e - h = 3$$

$$e = 3v - h - 3$$

■

- (b) Suppose you are given a quadrangulation with v vertices, h of which lie on the outer face. Use Euler's formula to derive the number of quadrangles q and the number of edges e in the quadrangulation as functions of v and h .

Solution: Total number of edges incident to all quadrangles:

$$4q = 2e - h$$

$$2e = 4q + h$$

Euler's formula (skipping steps shown in (a)):

$$v - e + q = 1$$

$$2v - 2e + 2q = 2$$

Replace $2e$ with $4q + h$ in the equation $2v - 2e + 2q = 2$:

$$2v - 4q - h + 2q = 2$$

$$2v - 2q - h = 2$$

$$2q = 2v - h - 2$$

$$q = v - \frac{h}{2} - 1$$

Plug q back into $v - e + q = 1$:

$$v - e + v - \frac{h}{2} - 1 = 1$$

$$2v - e - \frac{h}{2} = 2$$

$$e = 2v - \frac{h}{2} - 2$$

■

- (c) Explain why your answer to part (b) implies that no quadrangulation exists where the number of vertices on the outer face is odd.

Solution: If h was odd, then $\frac{h}{2}$ would not be an integer, therefore both the number of quadrangles and the number of edges would not be integers. We can't have a partial quadrangle or edge, so h must not be odd. ■

- (a) Compute the line equations of the two "crossing tangents", that is, the lines $\ell_1 : y = a_1x - b_1$ and $\ell_2 : y = a_2x - b_2$ that are both supporting lines for the convex hulls of each of P and Q such that P lies below ℓ_1 and above ℓ_2 and the reverse holds for Q . (Note that you are not given the hulls, just the point sets.) Your algorithm should run in $O(n + m)$ time.

Solution: To find the lines ℓ_1 and ℓ_2 , we will convert the problem into an upper and lower envelope problem and solve for the optimal vertex using linear programming. To accomplish this, we will convert the points in P and Q into their duals, swap their x and y coordinates, and then find the optimal vertices by minimizing y within the upper envelope and the maximizing y within the lower envelope. We begin by converting the points in P and Q into their duals.

$$P^* = \left\{ y = c_i x - d_i \mid \forall (c_i, d_i) \in P \right\}$$

$$Q^* = \left\{ y = c_i x - d_i \mid \forall (c_i, d_i) \in Q \right\}$$

From here the optimal vertices would reside within the left and right envelopes. To convert to upper and lower envelopes, we just swap the x and y coordinates.

$$P^* = \left\{ x = c_i y - d_i \mid \forall (c_i, d_i) \in P \right\}$$

$$= \left\{ y = \frac{x + d_i}{c_i} \mid \forall (c_i, d_i) \in P \right\}$$

$$Q^* = \left\{ x = c_i y - d_i \mid \forall (c_i, d_i) \in Q \right\}$$

$$= \left\{ y = \frac{x + d_i}{c_i} \mid \forall (c_i, d_i) \in Q \right\}$$

Now we can construct our linear programming problems.

ℓ_1 is a line above P and below Q . Therefore, its dual point would be below P^* and above Q^* . So we convert the lines in P^* into closed lower halfplanes $y \leq \frac{x + d_i}{c_i}$. And the lines in Q^* into closed upper halfplanes $y \geq \frac{x + d_i}{c_i}$. This way our optimal vertex for the dual of ℓ_1 will always fall equal to or below P^* and equal to or above Q^* . Our problem is then:

Maximize y

Subject to:

$$\left\{ y \leq \frac{x + d_i}{c_i} \mid \forall (c_i, d_i) \in P^* \right\}$$

$$\left\{ y \geq \frac{x + d_i}{c_i} \mid \forall (c_i, d_i) \in Q^* \right\}$$

ℓ_2 is a line below P and above Q , so we reverse the halfplanes. P^* would then be converted into closed upper halfplanes $y \geq \frac{x + d_i}{c_i}$ and Q^* would be closed lower halfplanes $y \leq \frac{x + d_i}{c_i}$. Our problem is then:

Minimize y

Subject to:

$$\left\{ y \geq \frac{x + d_i}{c_i} \mid \forall (c_i, d_i) \in P^* \right\}$$

$$\left\{ y \leq \frac{x + d_i}{c_i} \mid \forall (c_i, d_i) \in Q^* \right\}$$

Each problem will give us an optimal vertex, $\ell_1^* = (b_1, a_1)$ and $\ell_2^* = (b_2, a_2)$. After swapping the x and y values back ($\ell_1^* = (a_1, b_1)$ and $\ell_2^* = (a_2, b_2)$), we will find that their duals are the supporting lines for the convex hulls of P and Q .

$$\ell_1 : y = a_1x - b_1$$

$$\ell_2 : y = a_2x - b_2.$$

■

- (b) You have a cannon in $\mathbb{R}^2$. It has three controls labeled "a", "b", and "c". A projectile shot from this cannon travels along the parabolic arc $y = a + bx - cx^2$. You are asked to determine whether it is possible to adjust the controls so that the projectile travels above a set of n building tops, represented by a point set $P = \{p_1, \dots, p_n\}$ and beneath a set of m floating balloons, represented by a point set $Q = \{q_1, \dots, q_m\}$. Your algorithm should run in time $O(n+m)$. (Do not worry about the cannon's location. Just determine if there is some parabola along which the projectile can travel.)

Solution: To convert this problem into a form that we can solve with linear programming, we will introduce an error term ϵ to the parabolic equation. This error term represents the margin between the parabola and the points in P and Q . If there exists a margin $\epsilon > 0$, then we'll know that a parabola with margin ϵ exists that can separate P and Q . Since P is expected to be below the parabola, the points in P must be at least ϵ distance below the parabola. Since Q is expected to be above the parabola, the points in Q must be at least ϵ distance above the parabola. We will maximize ϵ subject to these constraints.

Maximize ϵ

Subject to:

$$\left\{ y_i \leq a + bx_i - cx_i^2 - \epsilon \mid \forall (x_i, y_i) \in P \right\}$$

$$\left\{ y_i \geq a + bx_i - cx_i^2 + \epsilon \mid \forall (x_i, y_i) \in Q \right\}$$

This linear programming problem will return ϵ . If $\epsilon > 0$, then there is a parabola between P and Q with at least ϵ distance to every point in P and Q . Therefore, it is possible to adjust the controls so that the projectile travels above P and beneath Q . If $\epsilon \leq 0$, then there will be some point in P or Q with at least ϵ distance across any parabola. Therefore, it is not possible to adjust the controls so that the projectile travels above P and beneath Q . ■

- (c) You are given a set of n halfplanes $H = \{h_1, \dots, h_n\}$. Each halfplane h_i is given as a pair (a_i, b_i) and consists of all the points (x, y) such that $y \leq a_i x - b_i$. Compute the axis-parallel square of the largest side length lying in $h_1 \cap \dots \cap h_n$ whose lower edge lies on the x -axis. If no such square exists, your algorithm should indicate this.

Solution: To find the axis-parallel square of the largest side length, we can treat the problem as a linear programming problem modified to find two optimal vertices instead of one. The vertices will be the horizontal bounds for the top edge of the square. Each vertex will be constrained by the halfplanes in H . In addition, we will have to add a couple more constraints to ensure that all sides of the square will have equal length.

x_1 and x_2 will be the x-values that bound the left and right edges of the square. ℓ will be the length of all sides and will also serve as the y-value of the top edge. The problem is then:

Maximize ℓ

Subject to:

$$\begin{aligned} & \left\{ \ell \leq a_i x_1 - b_i \mid \forall (a_i, b_i) \in H \right\} \\ & \left\{ \ell \leq a_i x_2 - b_i \mid \forall (a_i, b_i) \in H \right\} \\ & x_2 - x_1 \geq \ell \\ & x_1 < x_2 \end{aligned}$$

$x_2 - x_1 \geq \ell$ ensures that the width between the bounding halfplanes is at least the side length of the square. $x_1 < x_2$ ensures that x_1 is the left edge and x_2 is the right edge.

Solving this linear programming problem will either result in the optimal length of the square as well as the x-values that bound the top edge, or it will fail to return a solution, indicating that no such square exists. If a square exists we can calculate corners for it:

$$\begin{aligned} x_1 &\leq L \leq x_2 - \ell \\ x_1 + \ell &\leq R \leq x_2 \\ R - L &= \ell \\ (L, 0), (R, 0), (L, \ell), (R, \ell) \end{aligned}$$

■

- (a) Suppose we are given n points and create a subdivision as described above. Show that the resulting subdivision has size $O(n)$ (counting vertices, edges, and faces).

Solution: Each time we add a point to the subdivision, there are three possible cases:

- (a) The point is inside an existing face. In this case, we shoot bullets left, right, and up. Each of these creates a new vertex when it hits an edge. Add in the point itself and that's 4 new vertices. Each bullet creates a new edge so 3 new edges. There's now a face below the horizontal line and to the left and right of the vertical line. This adds 2 faces.

So in total we add 4 vertices, 3 edges, and 2 faces.

- (b) The point is on a horizontal edge. In this case, we split the edge into two edges and shoot a bullet up. The face is split into two faces. One new vertex is created at the end of the bullet path and a vertex is added for the new point.

So in total we add 2 vertices, 2 edges, and 1 face.

- (c) The point is on a vertical edge. In this case, we split the edge into two edges and shoot a bullet left and right. The left and right faces are split into two faces each. One new vertex is created at the end of each bullet path and a vertex is added for the new point.

So we add 3 vertices, 3 edges, and 2 faces.

In the worst case, we add 4 vertices, 3 edges, and 2 faces for each point. So the size of the subdivision is $(4 + 3 + 2) \cdot n = 9n = O(n)$. ■

- (b) Prove that for any $1 \leq i \leq n$, the expected number of new boxes created during the i th addition is $O(1)$.

Solution: Pulling heavily from Mount's notes because this is basically the same as the proof that the expected number of new trapezoids created during the i th addition to a trapezoidal map is $O(1)$.

Lemma: Consider the randomized incremental construction of a subdivision given points $P = \{p_1, \dots, p_n\}$, and let k_i denote the number of new boxes created when the i th point is added. Then $E[k_i] = O(1)$, where the expectation is taken over all possible permutations of the points as the insertion orders.

Proof: Let S_i denote the subdivision resulting from the insertion of the i th point. Because we are averaging over all permutations, among the i points in S_i , each point has an equal probability $\frac{1}{i}$ of being the last one added. For each of the points p , we want to count the number of boxes that would have been created if p had been the last point added (The worst case scenario for p).

We say that a box b depends on a point p if p would have caused b to be created had p been the last point inserted. We want to count the number of boxes that depend on each point and then compute the average over all points. If we let $\delta(b, p) = 1$ if b depends on p and 0 otherwise, then the expected value is:

$$E[k_i] = \sum_{p \in P_i} (\text{probability that } p \text{ is last}) \cdot (\text{number of boxes that depend on } p)$$

Where P_i is the set of points we've considered so far. We can rewrite this as:

$$\begin{aligned} E[k_i] &= \sum_{p \in P_i} \frac{1}{i} \left(\sum_{b \in S_i} \delta(b, p) \right) \\ &= \frac{1}{i} \sum_{p \in P_i} \sum_{b \in S_i} \delta(b, p) \\ &= \frac{1}{i} \sum_{b \in S_i} \sum_{p \in P_i} \delta(b, p) \end{aligned}$$

Each box is bounded by lines from at most 4 points. It may be less than 4 if the box is bounded by the bounding rectangle instead. The top and bottom of each box are formed by bullet paths going left or right from two points (or by the boundary rectangle of course). The left and right are formed by bullet paths going up from two points.

Therefore, $\sum_{p \in P_i} \delta(b, p) \leq 4$.

Since S_i consists of at most $2i + 1$ boxes (we start with 1 box and, as stated in part (a), add at most 2 boxes for each point), we have:

$$E[k_i] \leq \frac{1}{i} \sum_{b \in S_i} 4 = \frac{4}{i} |S_i| \leq \frac{4}{i} (2i + 1) = O(1)$$

■